

The logo for Oracle Academy is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font. Below it, the word "Academy" is written in a smaller, dark gray, sans-serif font. The entire logo is framed by a thin black border, with dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Java Fundamentals

4-4

Strings

ORACLE
Academy



Copyright © 2022, Oracle e/ou suas empresas afiliadas. Oracle, Java e MySQL são marcas comerciais registradas da Oracle Corporation e/ou de suas empresas afiliadas. Outros nomes podem ser marcas comerciais de seus respectivos proprietários.

Objetivos

- Esta aula abrange os seguintes tópicos:
 - Instanciar (criar) uma String
 - Descrever o que acontece quando uma String é modificada
 - Usar os operadores + e += de Strings concatenadas
 - Interpretar as sequências de escape em literais de String



Visão geral

- Esta aula abrange os seguintes tópicos:
 - Reconhecer a diferença entre uma String e um tipo de dados de caracteres primitivos
 - Testar Strings com o método `compareTo()` e `equals()`
 - Descrever por que o operador `==` nem sempre funciona quando a igualdade de Strings é testada
 - Usar métodos de Strings `length()`, `substring()`, `indexOf()` e `charAt()`

O que é uma String?

- Uma String é um objeto que contém uma sequência de caracteres
- Declarar e instanciar uma String é muito parecido com qualquer outra variável de objeto
- No entanto, há diferenças:
 - Elas podem ser instanciadas (criadas) sem usar a nova palavra-chave
 - Elas são imutáveis
 - Depois de instanciadas, elas são finais e não podem ser alteradas



Modificando uma String

- Tentar modificar uma String não a modifica, apenas cria um novo objeto de String
- Como um novo programador, você não notará essa diferença
- No entanto, torna-se importante em sistemas reais, nos quais o tempo de processamento é um elemento-chave na concepção do programa

Exemplo de Operações com Strings

```
public class StringOperations{
    public static void main(String[] args){
        String string1 = "Hello";
        String string2="Lisa";
        String string3="";           //empty String or null
        string3="How are you ".concat(string2);
        System.out.println("string3: "+ string3);
        //get length
        System.out.println("Length: "+ string1.length());
        //get substring beginning with character 0, up to, but
        //not including character 5
        System.out.println("Sub: "+ string3.substring(0,5));
        //uppercase
        System.out.println("Upper: "+string3.toUpperCase());
    } //fim do método main
} //fim da classe StringOperations
```

Modelo de Classe

- Use a seguinte classe para inserir exemplos nesta seção

```
import java.util.Scanner;  
  
public class StringPractice{  
    public static void main(String[] args){  
        //paste practice code here  
    }//fim do método main  
  
}//fim da classe StringPractice
```

Instanciando uma String

- Strings são tipos de referência de objeto
- Elas podem ser instanciadas de duas maneiras:
 - O novo operador
 - Literais de String
 - Não existe diferença entre as Strings a seguir
 - Ambos os métodos de instanciação criam objetos idênticos

```
String s1 = new String("abc"); // new operator
String s2 = "abc";             // String literals
```

Referências de String

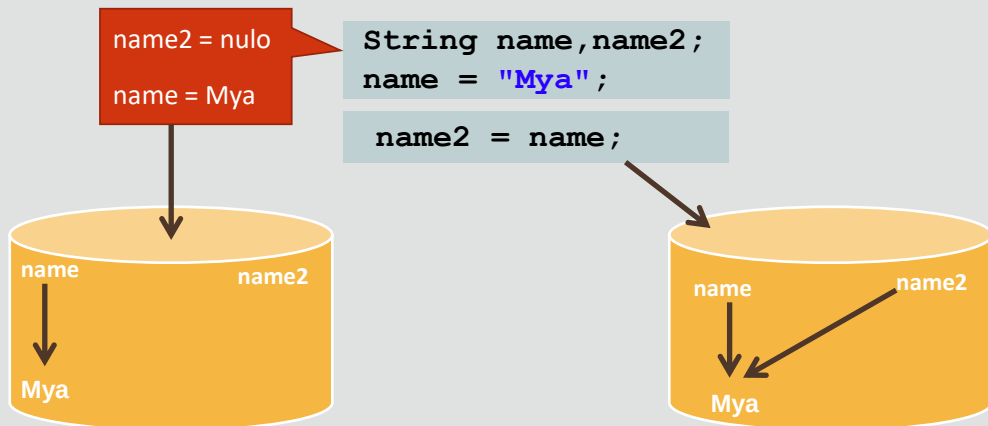
- Quando você cria uma referência a um Objeto ou uma String, o objeto não necessariamente já existe
- No código a seguir, visto que o nome não foi inicializado, o programa não compilará
- O nome da variável é um ponteiro nulo

```
String name;  
System.out.println("My name is " + name);
```

Nome = nulo

Referências de String

- Neste código, um objeto de String existe e o nome faz referência a ele
 - A referência name2 é nula
 - Como esta linha mudará a memória?



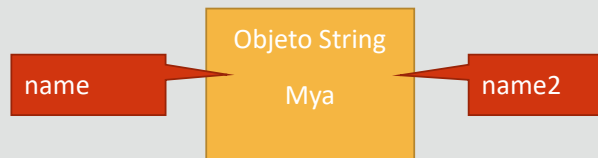
Uma referência, na verdade, armazena o endereço de onde o objeto está na memória. `name` e `name2` têm o mesmo endereço armazenado e, desse modo, apontam para o mesmo objeto, `"Mya"`, na memória.

Referências de String

- Executar esta linha de código mudará a referência name2

```
name2 = name;
```

- A referência name2 agora fará referência ao mesmo objeto que o nome

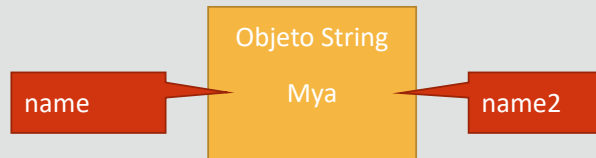


Referências de String

- Executar estas linhas de código cria duas instâncias da String Mya na memória

```
String name, name2;  
name = "Mya";  
name2 = "Mya";
```

- Aqui o compilador JVM decide poupar espaço e armazenar somente um Objeto de String que contenha a String Mya

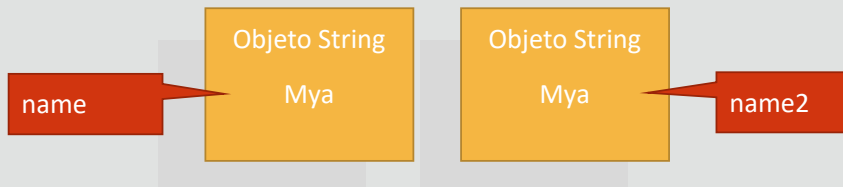


Referências de String

- Se executarmos um programa muito semelhante e pedirmos ao usuário para digitar Strings, teremos um resultado diferente

```
Scanner in= new Scanner(System.in);  
String name, name2;  
name=in.next();  
name2=in.next();
```

- Se o usuário digitar Mya para ambas as Strings, o compilador criará dois objetos de String diferentes

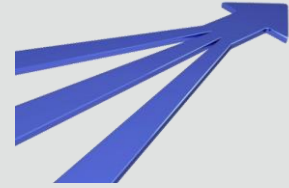


Referências de String

- É muito difícil prever quando o Java criará uma nova String ou usará um objeto existente para uma String que seja idêntica à nova String



Concatenação de Strings



- Concatene duas strings com os operadores + e +=
- O símbolo + é usado para concatenar duas strings
- O símbolo += é usado para concatenar duas strings e atribuí-las a si mesmas, tudo em uma operação

```
String s1 = "This is a ";  
String s2 = "string";  
String s3 = s1 + s2;  
String s4 = "This is a " + s2;  
String s1 += s2;
```

Concatenar significa vincular itens em uma cadeia ou série.
Concatenação de Strings é a vinculação de duas ou mais strings.



ORACLE
Academy

JF 4-4
Strings

Copyright © 2022, Oracle e/ou suas empresas afiliadas. Oracle, Java e MySQL são marcas comerciais registradas da Oracle Corporation e/ou de suas empresas afiliadas. Outros nomes podem ser marcas comerciais de seus respectivos proprietários.

16

Experimente o operador += em uma variável do tipo inteiro (X += 10;). Além disso, pratique o uso dos operadores -=, *=, /= e até mesmo %= para ver quais resultados eles retornam com valores inteiros.

Manipulação de Strings

- Cada vez que uma String é alterada, uma nova String é criada em um novo local da memória e a referência é movida para o novo local
- Alguns Strings apontarão para o mesmo objeto de referência na memória
- Fazer uma modificação em uma String que esteja apontando para a referência compartilhada não fará uma modificação nas outras
- A que você modifica aponta para o novo endereço de memória que contém a string nova/modificada e as outras ainda apontarão para a original

String Concatenation Example

- Qual será a saída de s1, s2, s3 e s4 e no final destas instruções?

```
String s1 = "This is a ";  
String s2 = "string";  
String s3 = s1 + s2;  
String s4 = "This is a " + s2;  
String s1 += s2;  
System.out.println("s1: " + s1);  
System.out.println("s2: " + s2);  
System.out.println("s3: " + s3);  
System.out.println("s4: " + s4);
```

- Digite essas instruções em um programa Java e verifique os resultados
- Seu palpite estava correto?

Sequências de Escape \

- As sequências de escape em literais de string permitem ao usuário adicionar caracteres que seriam mal-interpretados pelo compilador
- Por exemplo, para incluir aspas duplas em sua String, o compilador interpretaria as aspas duplas como o início ou o fim da String, em vez de incluí-las
- Use a sequência de escape \" para incluir aspas duplas

```
String s1 = "Isso é um exemplo de escape.  
           \n E agora estamos em uma nova linha. \n \t Isso é uma tabulação."  
String s2 = "\"This is a quote \"";  
System.out.println(s1);  
System.out.println(s2);
```

Sequências de Escape Comuns em Java

- Veja aqui uma lista de algumas sequências de escape comuns em Java

Sequência de Escape	Representação	Representação OctalEscape Alternativa
<code>\"</code>	Aspas duplas	<code>\u0022</code>
<code>\'</code>	Aspas simples	<code>\u0027</code>
<code>\\</code>	Barra invertida	<code>\u005c</code>
<code>\t</code>	Tabulação horizontal	<code>\u0009</code>
<code>\n</code>	Alimentação de linha	<code>\u000a</code>
<code>\b</code>	Backspace	<code>\u0008</code>
<code>\r</code>	Retorno de carro	<code>\u000d</code>
<code>\f</code>	Avanço de página	<code>\u000c</code>
Octal Escape	Valor octal	<code>\u0000</code> to <code>\u00ff</code>

Método compareTo

- Existem métodos a serem usados ao comparar Strings
 - Método: `s1.compareTo(s2)`
 - Deve ser usado ao tentar encontrar a ordem lexicográfica de duas strings
 - Retorna um valor inteiro
 - Se `s1` for menor que `s2`, um `int < 0` será retornado
 - Se `s1` for igual a `s2`, `0` será retornado
 - Se `s1` for maior que `s2`, um `int > 0` será retornado



O método `compareTo()` é necessário em algoritmos de classificação.

Método equals

- Existem métodos a serem usados ao comparar Strings
 - Método: `s1.equals(s2)`
 - Deve ser usado quando você só deseja saber se as duas strings são iguais
 - Retorna um valor booleano
 - Se `true` for retornado, `s1` será igual a `s2`
 - Se `false` for retornado, `s1` não será igual a `s2`



Exemplo de Métodos compareTo e equals

- O que este método compareTo exibirá?

```
String s1 = "abc";  
String s2 = "cde";  
System.out.println(s1.compareTo(s2));
```

- O que este método equals exibirá?

```
String s1 = "abc";  
String s2 = "ABC";  
System.out.println(s1.equals(s2));
```

Solução de Exemplo de Métodos compareTo e equals

- Este método compareTo exibirá um int menor que 0

```
String s1 = "abc";  
String s2 = "cde";  
System.out.println(s1.compareTo(s2));
```

- Este método equals exibirá false

```
String s1 = "abc";  
String s2 = "ABC";  
System.out.println(s1.equals(s2));
```

Comparando Strings com ==

- Variáveis primitivas podem ser comparadas com ==
- Este método de comparação nem sempre funciona com Strings e deve ser evitado a menos que esteja tentando comparar a localização de endereços da memória de dois objetos de String
- == compara valores de referência, não valores
- Será verdadeiro somente se dois objetos de String estiverem apontando para o mesmo objeto de referência

```
String s1 = "This is a String.";
String s2 = new String("This is a String.");
String s3 = "String.";
String s4 = "This is a " + s3;
System.out.println(s1 == s2);
System.out.println(s1 == s4);
```

Nunca use == para ver se dois objetos String têm o mesmo valor. Isso produzirá erros lógicos, ou seja, o código é compilado e executado, mas nem sempre fornece a resposta correta. == compara os endereços onde o objeto está na memória, mas não os valores reais nesse local. Ao usar ==, você pode ter dois objetos String com o mesmo valor retornarem falso.

Comparando Strings com ==

- Qual exibirá true no exemplo a seguir?
 - s1 e s2 apontará para o mesmo objeto
 - A Java perceberá que os objetos são o mesmo e fará referência a ele
 - s5 não apontará para o mesmo objeto que s1
 - Ela é “criada” a partir de uma combinação de duas Strings e, portanto, um novo objeto será criado

```
String s1 = "This is a String.";
String s2 = "This is a String.";
String s3 = "This is ";
String s4 = "a String.";
String s5 = s3 + s4;
```

```
System.out.println(s1 == s2);
System.out.println(s1 == s5);
```

Método Útil de String: length()

- Método: s1.length()
- Retorna o comprimento ou o número de caracteres em s1 como um int
- O comprimento da string é um método acessador chamado em um objeto de String que retornará a variável de comprimento da String

```
String s1 = "This is a string.";
int n = s1.length();
//n is 17 because s1 has 17 characters
```

Uma string em Java pode ter comprimento de 0 - "". Em geral, isso é chamado de string vazia ou nula.

Cada Caractere de String tem um Índice

- Visto que as Strings são uma representação de uma sequência de caracteres, cada caractere de uma String tem um índice
- Um índice é uma referência de posição
- O primeiro caractere de uma String tem um índice 0, o segundo tem um índice 1 e assim por diante até o final da String



ORACLE
Academy

JF 4-4
Strings

Copyright © 2022, Oracle e/ou suas empresas afiliadas. Oracle, Java e MySQL são marcas comerciais registradas da Oracle Corporation e/ou de suas empresas afiliadas. Outros nomes podem ser marcas comerciais de seus respectivos proprietários.

28

Conceitualmente, isso é similar a Matrizes, que serão abordadas mais adiante neste curso.

Método Útil de String: substring()

- `s2.substring(int beginIndex)`
 - Retorna parte da string `s2` do `beginIndex` no final da String
- `s2.substring(int beginIndex, int endIndex)`
 - Retorna parte da string `s2` do `beginIndex` no `endIndex`, mas não inclui o caractere em `endIndex`
 - O que é exibido como resultado deste segmento de código?

```
String s1 = "I eat apples";  
String s2 = "Bananas are my favorite fruit";  
System.out.println(s1.substring(6)  
                    +" "+s2.substring(8,23) );
```

Métodos Úteis de String: indexOf e charAt

- Método: **`s3.indexOf(char c)`** ;
 - Retorna o valor de índice da primeira ocorrência de c em String s3
- O parâmetro não precisa ser um caractere, ele pode ser uma String, por exemplo: **`s3.indexOf("the")`** ;
- Método: **`s4.charAt(int index)`** ;
 - Retorna o caractere da String localizada no índice especificado como parâmetro
 - O índice pode ser um valor inteiro de 0 a **`s4.length() - 1`**

Usando Métodos de String Exemplo 1

- Escreva o código Java que usará um endereço de e-mail como entrada no formato de uma String
 - Esse método retornará o domínio
 - Por exemplo, se o usuário digitar john@oracle.com, o método retornará oracle.com
 - Para solucionar esse problema, pesquise primeiro o caractere '@' na String
 - Depois, a saída da parte da substring da String inicial logo após essa posição

```
String domain="";           //no space between the " and "  
int position=email.indexOf('@');  
domain=email.substring(position+1);  
System.out.println(domain);
```

Terminologia

- Os principais termos usados nesta lição foram:
 - Concatenação
 - Sequências de escape
 - Instanciar
 - Objeto de referência
 - Objeto String
 - Métodos de String (compareTo e equals)

Resumo

- Nesta aula, você deverá ter aprendido a:
 - Instanciar (criar) uma String
 - Descrever o que acontece quando uma String é modificada
 - Usar os operadores + e += de Strings concatenadas
 - Interpretar as sequências de escape em literais de String
 - Reconhecer a diferença entre uma String e um tipo de dados de caracteres primitivos
 - Testar Strings com o método compareTo() e equals()
 - Descrever por que o operador == nem sempre funciona quando a igualdade de Strings é testada
 - Usar métodos de Strings length(), substring(), indexOf() e charAt()



